# Système d'exploitation
# CM0

Pierre Rousselin

Université Sorbonne Paris Nord
L2 informatique et double-licence
janvier 2026

Organisation du cours

Programmer en langage C dans ce cours

Objectifs du cours

Types entiers en machine et en C

# 12 semaines de dur labeur

- 1h30 de CM le lundi dès le lever
  pour commencer la semaine du bon pied !
- 1h30 de TD à la suite le lundi
  pour réfléchir sans écran ni ordi !
- 1h30 de TP le vendredi
  et on écrit du joli C
  pour finir la semaine en beauté !

Ah, vraiment, la programmation système,
c'est ça qu'on aime !
Ah, vraiment, la programmation réseau,
c'est ça qu'est beau !

Institut GALILÉE
Université Sorbonne Paris Nord

# ANNEE UNIVERSITAIRE 2025-2026

## LICENCE 2ème année

| SEMAINE | 01-sept 05-sept | 08-sept 12-sept | 15-sept 19-sept | 22-sept 26-sept | 29-sept 03-oct | 06-oct 10-oct | 13-oct 17-oct |
|---|---|---|---|---|---|---|---|
| Lundi | | | | | | | |
| Mardi | | | | | | | |
| Mercredi | | | 1 | 2 | 3 | 4 | 5 |
| Jeudi | | | | | | | |
| Vendredi | | | | | | | |

| SEMAINE | 20-oct 24-oct | 27-oct 31-oct | 03-nov 07-nov | 10-nov 14-nov | 17-nov 21-nov | 24-nov 28-nov | 01-déc 05-déc |
|---|---|---|---|---|---|---|---|
| Lundi | | | | | | | |
| Mardi | | | | | | | |
| Mercredi | 6 | VACANCES D'AUTOMNE | 7 | Partiels 1 N3 | 8 | 9 | 10 |
| Jeudi | | | | | | | |
| Vendredi | | | | | | | |

| SEMAINE | 08-déc 12-déc | 15-déc 19-déc | 22-déc 26-déc | 29-déc 02-janv | 05-janv 09-janv | 12-janv 16-janv | 19-janv 23-janv |
|---|---|---|---|---|---|---|---|
| Lundi | | | | | | | |
| Mardi | | | | | | | |
| Mercredi | 11 | 12 | VACANCES DE NOEL | | PARTIELS 2 N3 | Semaine Orientation, Sciences et Culture | 1 |
| Jeudi | | | | | | | |
| Vendredi | | | | | | | |

| SEMAINE | 26-janv 30-janv | 02-févr 06-févr | 09-févr 13-févr | 16-févr 20-févr | 23-févr 27-févr | 02-mars 06-mars | 09-mars 13-mars |
|---|---|---|---|---|---|---|---|
| Lundi | | | | | | | |
| Mardi | | | | | | | |
| Mercredi | 2 Jury N3 | 3 Jury N3 | 4 | 5 | 6 | VACANCES D'HIVER | PARTIELS 1 N4 |
| Jeudi | | | | | | | |
| Vendredi | | | | | | | |

| SEMAINE | 16-mars 20-mars | 23-mars 27-mars | 30-mars 03-avr | 06-avr 10-avr | 13-avr 17-avr | 20-avr 24-avr | 27-avr 01-mai |
|---|---|---|---|---|---|---|---|
| Lundi | | | | | | | |
| Mardi | | | | | | | |
| Mercredi | 7 | 8 | 9 | 10 | 11 | VACANCES DE PRINTEMPS | VACANCES DE PRINTEMPS |
| Jeudi | | | | | | | |
| Vendredi | | | | | | | |

| SEMAINE | 04-mai 08-mai | 11-mai 15-mai | 18-mai 22-mai | 25-mai 29-mai | 01-juin 05-juin | 08-juin 12-juin | 15-juin 19-juin |
|---|---|---|---|---|---|---|---|
| Lundi | | | | | | | |
| Mardi | | | | | | | |
| Mercredi | 12 | Révisions | PARTIELS 2 N4 (DAPS) | Soutien N3 | Soutien N3 Jury N4 | 2nde chance N3 | Soutien N4 |
| Jeudi | | | | | | | |
| Vendredi | | | | | | | |

| SEMAINE | 22-juin 26-juin | 29-juin 03-juil | 06-juil 10-juil | 13-juil 17-juil | 20-juil 24-juil |
|---|---|---|---|---|---|
| Lundi | | | | JURY L2 2nd chance | |
| Mardi | | | | | |
| Mercredi | 2de chance N4 | 2de chance N4 | JURY L2 2nd chance | | |
| Jeudi | | | | | |
| Vendredi | | | | | |

# Note du module

- Évaluation continue (EvC) : QCM réguliers en TD ou TP, ...
- Partiel 1 ($\mathrm{Pa}_1$) et Partiel 2 ($\mathrm{Pa}_2$) qui durent 3h chacun
- Un projet (Pr) en fin de module
- Note finale avant rattrapage :

$$\mathrm{NF} = \max\left(\frac{\mathrm{EvC} + 2\mathrm{Pa}_1 + \mathrm{Pr} + 4\mathrm{Pa}_2}{8}, \frac{\mathrm{Pa}_1 + 2\mathrm{Pa}_2}{3}\right)$$

- Si $\mathrm{NF} < 12$, possible de rattraper, note finale après rattrapage (Pa) éventuel :

$$\mathrm{ND} = \max\left(\mathrm{Pa}, \frac{\mathrm{EvC} + \mathrm{Pr} + 6\mathrm{Pa}}{8}\right)$$

# Ce module est exigeant

- Présence en CM très conseillée !
- Travaillez le cours régulièrement.
- Chaque semaine (ou presque), une fiche d'auto-évaluation comportant une dizaine de questions vous permet de faire le point sur ce que vous connaissez du CM.
- QCM : en TD (dans ce cas, sur le CM/TD/TP de la semaine précédente) ou en TP (dans ce cas sur le CM/TD de la semaine en cours). Utiliser la fiche d'auto-évaluation pour le préparer.
- Programmez pendant les séances de TP mais aussi en dehors.
- Reprenez les exercices du TD d'une fois sur l'autre et faîtes les exercices demandés par vos chargés de TD.
- Pour préparer un partiel, essayez de faire celui de l'année dernière et étudiez ensuite la correction.
- Identifiez vos lacunes (en C, en calcul, ...) et comblez-les petit à petit.
- Vous ne pourrez pas valider ce module sans avoir un très bon niveau en C à la fin !

# Système Unix

- Si vous n'avez pas de machine, c'est le moment de s'en procurer une (maison des étudiants, vieux portable d'occasion, ...).
- Vous pouvez économiser en n'achetant pas de système d'exploitation et en installant Linux.
- Système de type Unix : Linux (libre) ou macOs (propriétaire) ou autre mais pas Windows.

# Si vous êtes sous Windows

- Très conseillé : installez Linux (Debian, Fedora, etc, peu importe la distribution) en *dual-boot* (ou même en enlevant Windows).
- Un peu moins bien : installez Linux sur une machine virtuelle.
- Encore un peu moins bien : installez Linux avec le *Windows Subsystem for Linux* (WSL 2). Dans ce cas, ce sera votre problème si des appels systèmes ne sont pas implémentés.

# Si vous êtes sous macOS

- Système Unix basé sur BSD (*Berkeley Software Distribution*)
- en théorie supporté dans ce cours mais
- je n'ai pas le temps de tester sur macOS, s'il y a des choses qui ne fonctionnent pas comme prévu, le dire (*mail*, question pendant ou après le cours, etc).
- Installez un gestionnaire de paquets (par exemple *homebrew*[1]) pour pouvoir installer les paquets manquants.
- Les sémaphores anonymes ne sont pas implémentés, vous devrez utiliser des sémaphores nommés à la place. On vous dira comment faire.

1. `https://brew.sh/`

Organisation du cours

Programmer en langage C dans ce cours

Objectifs du cours

Types entiers en machine et en C

# C

- Oui, c'est encore du C. (Super ! ou Zut ! ?)
- Tous les Unix (Linux, MacOS, ...) sont écrits en C, c'est le langage qui a été inventé pour ça et qui est le plus naturel pour ce module.
- Performances et simplicité inégalables, mais en effet, ça a un prix : il faut tout faire à la main.

# Édition de texte

- Vous utilisez l'éditeur de texte de votre choix. À ce stade, un éditeur puissant est recommandé mais pas obligatoire.
- Vous devez être autonome avec votre éditeur, les enseignants peuvent éventuellement vous aider s'ils sont compétents mais ne sont pas tenus de l'être (par exemple, il est inutile de me poser des questions sur `emacs`).
- Votre éditeur devrait vous aider à indenter votre code et à coder plus vite, mais pas vous remplacer.
- Toute aide d'intelligence artificielle générative (par exemple *copilot* ou interroger directement *ChatGPT*) est interdite. Problèmes éthiques [2], environnementaux [3], sociaux et pédagogiques. Il arrive également souvent que les réponses « probables » écrites par IA soient incorrectes factuellement.

2. https://www.fsf.org/news/publication-of-the-fsf-funded-white-papers-on-questions-around-copilot
3. https://news.mit.edu/2025/explained-generative-ai-environmental-impact-0117

# Compilation

Vous pouvez utiliser le compilateur de votre choix, du moment qu'il respecte le standard C99.
A priori, il s'agit de `gcc` (par défaut sur GNU/Linux) ou `clang` (par défaut sur macOS).
Ces deux compilateurs proposent les options suivantes :

- `-Wall` pour *all Warnings*, en fait pas tous, seulement ceux qui mettent à peu près tout le monde d'accord ; à utiliser obligatoirement ; sauf très rares exceptions votre code doit compiler sans warnings avec `-Wall`.
- `-Wvla` pour émettre un *warning* en cas d'utilisation de *Variable Length Array* qu'on interdit dans ce cours (voir plus loin).
- `-std=c99` pour s'assurer qu'on est conforme au standard C99.

# V'là les vla

- Les *Variable Length Array* sont des tableaux de taille fixe mais pas nécessairement connue à la compilation.
- Souvent stockés dans la pile.
- Introduits (et controversés) en C99, ils deviennent optionnels en C11 (et de nouveau obligatoires en C23...)
- Ils n'apportent pas grand chose par rapport aux
  - tableaux C (dans la pile, de taille fixe connue à la compilation)
  - tableaux « dynamiques » (dans le tas, de taille fixe non nécessairement connue à la compilation).
- Linus Torvalds ne les autorise pas non plus dans le noyau Linux.

```c
int main(void)
{
    size_t n;
    printf("Taille du tableau : ");
    scanf("%zu", &n);
    int tab[n]; /* VLA, interdit dans ce cours ! */
    memset(tab, 2, n * sizeof(int));
    size_t i;
    for (i = 0; i < n; ++i)
        printf("tab[%zu] = %d\n", i, tab[i]);
    return 0;
}
```

# Valgrind

- Vous devez utiliser `valgrind` avant de rendre du code.
- Entre autres choses, `valgrind` peut détecter :
  - des fuites de mémoire ;
  - l'utilisation de variables non définies ;
  - l'accès à une case mémoire en dehors des limites d'un tableau ;
  - et peut très souvent aider à trouver l'origine des *Segmentation Fault*.
- Utiliser l'option de compilation `-g` pour intégrer les symboles de débogage à vos binaires (associer des noms de fichiers, de fonctions, numéros de lignes,... à des instructions)
- Il arrive (très rarement) que `valgrind` produise des faux positifs.

# valgrind, exemple 1 (test sur variable non initialisée)

```c
#include <stdio.h>
int main(void)
{
        int n;
        if (n == 0) { /* oops! */
                printf("plop\n");
        }
        return 0;
}
```

Dans ce cas simple, le compilateur nous prévient déjà.

```
$ gcc -g -Wall -Wvla -std=c99 valgrind-non-def.c
valgrind-non-def.c: Dans la fonction « main »:
valgrind-non-def.c:6:12: attention: « n » est utilisé
sans avoir été initialisé
[-Wuninitialized]
    6 |         if (n == 0) { /* oops! */
      |            ^
valgrind-non-def.c:5:13: note: « n » a été déclaré ici
    5 |         int n;
      |             ^
```

```
$ valgrind ./a.out
==760856== Conditional jump or move depends on uninitialised
           value(s)
==760856==    at 0x401132: main (valgrind-non-def.c:6)
```

# valgrind, exemple 2 (fuite et accès après un bloc)

```c
#include <stdlib.h>
void foo(int *bar)
{
        bar[3] = -12;
}
int main(void)
{
        int *baz = malloc(2 * sizeof(int));
        foo(baz);
        return 0;
}
```

```
$ gcc -g -Wall -Wvla -std=c99 valgrind-out-of-bounds.c
$ valgrind ./a.out
==763694== Command: ./a.out
==763694== Invalid write of size 4
==763694==    at 0x401136: foo (valgrind-out-of-bounds.c:4)
==763694==    by 0x401160: main (valgrind-out-of-bounds.c:10)
==763694==  Address 0x4a7804c is 4 bytes after a block of size 8 alloc'd
==763694==    at 0x4841866: malloc (vg_replace_malloc.c:446)
==763694==    by 0x401150: main (valgrind-out-of-bounds.c:9)
==763694== HEAP SUMMARY:
==763694==     in use at exit: 8 bytes in 1 blocks
==763694==   total heap usage: 1 allocs, 0 frees, 8 bytes allocated
==763694== LEAK SUMMARY:
==763694==    definitely lost: 8 bytes in 1 blocks
```

# Style

- Votre code doit être indenté de façon cohérente.
- J'utilise (à peu près) le style du noyau Linux `https://www.kernel.org/doc/html/v4.10/process/coding-style.html` qui lui-même est basé sur « Le langage C » de Kernighan et Ritchie.
- Vous pouvez utiliser le programme `indent`, par exemple de la façon suivante :

  ```
  $ indent -kr -i8 mon_prog.c
  ```

- mais le mieux est de toujours garder son code bien indenté.
- Les commentaires sont utilisés :
  - pour spécifier les fonctions à leurs déclarations
  - pour préciser l'intention (le « quoi »), mais en général pas pour expliquer l'implémentation (le « comment »). La plupart du temps, si vous pensez qu'il faut expliquer l'implémentation, c'est plutôt que vous devez rendre le code plus clair.
- Parfois on adopte volontairement des pratiques peu recommandées (mono-fichier, variables globales, ...) lorsque cela simplifie le travail de pédagogie.

# Yolo, World!

```
#include <stdio.h>
    int main(void    )
{ printf(
                    "Yolo, World!")            ;
        return



                0;
                            }
```

```
$ indent -kr -i8 yolo.c -o better-yolo.c
```

```
#include <stdio.h>
int main(void)
{
    printf("Yolo, World!");
    return 0;
}
```

(Remarque : faute de place, sur les transparents, la tabulation est représentée par 4 espaces, contre 8 en fait dans mon éditeur)

# Adopter de bons réflexes en TP (et chez soi)

- Compiler régulièrement et tenir compte de la sortie du compilateur.
- Une fois que le code compile, tester le programme.
- En cas d'erreur de segmentation ou de comportement aberrant, utiliser `valgrind` sans attendre qu'un chargé de TP vous demande de le faire.
- Si vous ne trouvez toujours pas l'origine de l'erreur, vous pouvez demander de l'aide... mais essayez tout de même :
  - des affichages [4]
  - faire la trace à la main sur votre instance de test
  - éventuellement un débogueur comme `gdb`
- Il faut aussi accepter le fait que déboguer un programme est long et difficile.

4. encore que... dans les cas subtils, les affichages peuvent cacher une erreur !

# Adopter de bons réflexes en TP (et chez soi)

- Compiler régulièrement et tenir compte de la sortie du compilateur.
- Une fois que le code compile, tester le programme.
- En cas d'erreur de segmentation ou de comportement aberrant, utiliser `valgrind` sans attendre qu'un chargé de TP vous demande de le faire.
- Si vous ne trouvez toujours pas l'origine de l'erreur, vous pouvez demander de l'aide... mais essayez tout de même :
  - des affichages [4]
  - faire la trace à la main sur votre instance de test
  - éventuellement un débogueur comme `gdb`
- Il faut aussi accepter le fait que déboguer un programme est long et difficile.

Ça ne sert à rien d'appeler un enseignant pour lui demander « C'est bon ? » si vous n'avez même pas essayé de compiler et d'exécuter !

4. encore que... dans les cas subtils, les affichages peuvent cacher une erreur !

Organisation du cours

Programmer en langage C dans ce cours

Objectifs du cours

Types entiers en machine et en C

# Système

Objectifs :

- Comprendre et savoir utiliser les différentes abstractions fournies par les (noyaux des) systèmes d'exploitation Unix : processus, fichiers, *threads*, *sockets*.
- Comprendre les problèmes liés au partage des tâches et à leurs synchronisation : situations de compétition (*race conditions*) et interblocage (*deadlock*). Savoir utiliser sémaphores et mutex.
- Avoir une image mentale, certes simpliste et un peu fausse mais éclairante, de ce que fait le noyau pour implémenter ces abstractions : ordonnancement des tâches, organisation de la mémoire, ...
- Écrire du joli C.

# Entrées/Sorties

Objectif principal : savoir écrire et surtout lire des octets dans un fichier qui peut être

- un fichier normal (facile)
- un tube (plus dur)
- le terminal
- une *socket* pour communiquer avec un autre processus sur la même machine ou sur un hôte distant
- nécessite d'être extrêmement rigoureux sur la représentation des données (entiers en binaire, texte, fins de lignes, etc)
- On écrira un programme client/serveur en C en utilisant l'API [5] standard POSIX [6].

5. *Application Programming Interface*
6. *Portable Operating System Interface (for UniX)*

# Prérequis

- Vous devriez déjà être un peu à l'aise en ligne de commande sur un système de type Unix.
- Vous devriez déjà avoir un bon niveau en C.
- Vous devriez déjà savoir calculer avec des puissances de 2, savoir ce qu'est la représentation en hexadécimal, etc.

# Prérequis

- Vous devriez déjà être un peu à l'aise en ligne de commande sur un système de type Unix.
- Vous devriez déjà avoir un bon niveau en C.
- Vous devriez déjà savoir calculer avec des puissances de 2, savoir ce qu'est la représentation en hexadécimal, etc.

Mais...

- On n'hésitera pas à faire des rappels si besoin, voire des feuilles d'exercices en plus de type « remédiation » pour aider les étudiants qui en ont besoin à combler leurs lacunes.
- Accrochez-vous et aidez-nous à vous aider.
- N'hésitez pas à poser des questions, même (surtout !) en CM !

Organisation du cours

Programmer en langage C dans ce cours

Objectifs du cours

Types entiers en machine et en C

# *Bits and bytes*

- Dans toutes les machines modernes, l'unité atomique d'information est le *bit* qui n'a que deux valeurs possibles
- souvent notées 0 ou 1.
- On peut imaginer la mémoire (RAM) de la machine comme une longue suite finie de bits...
- regroupés par paquets de 8 en *octets* (*byte*).
- On fait l'hypothèse (vraie en pratique en 2026) que cette mémoire est *byte-addressable*, c'est-à-dire que chaque octet a une adresse (un numéro) dans la mémoire qui permet d'y accéder.

# Nombre de valeurs possibles

- Sur un octet :

# Nombre de valeurs possibles

- Sur un octet : $2^8 = 256$ octets différents.
- Sur deux octets :

# Nombre de valeurs possibles

- Sur un octet : $2^8 = 256$ octets différents.
- Sur deux octets : $2^{16} = 65536$ valeurs différentes.
- Sur 10 bits :

# Nombre de valeurs possibles

- Sur un octet : $2^8 = 256$ octets différents.
- Sur deux octets : $2^{16} = 65536$ valeurs différentes.
- Sur 10 bits : $2^{10} = 1024 \approx 10^3$
- Sur quatre octets :

# Nombre de valeurs possibles

- Sur un octet : $2^8 = 256$ octets différents.
- Sur deux octets : $2^{16} = 65536$ valeurs différentes.
- Sur 10 bits : $2^{10} = 1024 \approx 10^3$
- Sur quatre octets : $2^{32} = (2^{10})^3 \times 2^2 \approx 4 \times 10^9$ (4 milliards, deux fois moins que le nombre d'êtres humains).

# Signitude (*signedness*)

Au départ (norme C89 et avant), seuls les types **int** et **char** existent avec des *modificateurs de types* : pour la longueur (slide suivant) ou le caractère signé ou non signé. Pour la signitude[7] :

- **signed** (signé) pour un type pouvant représenter des valeurs positives et négatives
- **unsigned** (non signé) pour un type ne représentant que des valeurs positives (ou nulles).
- Pour **int**, il est signé par défaut, donc **signed int** est synonyme de **int**.
- Pour **char**, cela dépend de la machine et/ou du compilateur. Donc **char** désigne soit **signed char**, soit **unsigned char**. La seule contrainte est qu'il doit pouvoir représenter les valeurs 0 à 127.

7. J'ai inventé ce mot, faute d'équivalent français acceptable.

# Taille

Constante de l'univers : un **char** est toujours sur un octet.

$$e^{i\pi} = -\texttt{sizeof(char)}$$

Pour les **int** et seulement pour eux, on peut utiliser un modificateur de taille avec :

- **short**, au moins sur 2 octets
- **long**, au moins sur 4 octets
- **long long** (depuis C99), au moins sur 8 octets
- Les **int** (sans modificateur de taille) sont quelque part entre les **short** et les **long**.
- Sans précision de signitude, c'est signé par défaut.
- Si on omet un type, c'est **int**.
- Par exemple **unsigned** signifie **unsigned int** ; **long** signifie **long int**, et **signed** est un synonyme de **int**.

# Un type pour les tailles

- L'opérateur `sizeof` permet de connaître la taille (en nombre d'octets) d'un type ou d'une variable.
- Son résultat est de type `size_t` qui est un type entier non signé permettant de décrire toutes les tailles. Il faut pouvoir au minimum numéroter tous les octets de l'espace mémoire d'un processus. Ce type dépend donc de la machine.
- C'est le compilateur qui est chargé de définir `size_t` avec un des types entiers (par exemple, `size_t` peut être un alias de type pour `unsigned long`).
- Spécificateur de format `"%zu"` (moyen mnémotechnique : `z` comme dans *siZe*, `u` pour *unsigned*) pour l'affichage et la saisie.

# Tailles des entiers

```c
#include <stdio.h>

int main(void)
{
    printf("Taille de short : %zu\n", sizeof(short));
    printf("Taille de int : %zu\n", sizeof(int));
    printf("Taille de long : %zu\n", sizeof(long));
    printf("Taille de long long : %zu\n", sizeof(long long));
    printf("Taille de size_t : %zu\n", sizeof(size_t));
    return 0;
}
```

# Tailles des entiers

```c
#include <stdio.h>

int main(void)
{
    printf("Taille de short : %zu\n", sizeof(short));
    printf("Taille de int : %zu\n", sizeof(int));
    printf("Taille de long : %zu\n", sizeof(long));
    printf("Taille de long long : %zu\n", sizeof(long long));
    printf("Taille de size_t : %zu\n", sizeof(size_t));
    return 0;
}
```

Sortie sur ma machine (x86_64) :

```
Taille de short : 2
Taille de int : 4
Taille de long : 8
Taille de long long : 8
Taille de size_t : 8
```

# Connaître ses limites

```c
#include <stdio.h>
#include <limits.h>

int main(void)
{
    printf("max des char : %d\n", CHAR_MAX);
    printf("min des char : %d\n", CHAR_MIN);
    printf("max des signed char : %hhi\n", SCHAR_MAX);
    printf("min des signed char : %hhi\n", SCHAR_MIN);
    printf("max des unsigned char : %hhu\n", UCHAR_MAX);
    printf("max des short : %hi\n", SHRT_MAX);
    printf("min des short : %hi\n", SHRT_MIN);
    printf("max des int : %d\n", INT_MAX);
    printf("min des int : %d\n", INT_MIN);
    printf("max des unsigned : %u\n", UINT_MAX);
    return 0;
}
```

# Connaître ses limites (sortie)

```
max des char : 127
min des char : -128
max des signed char : 127
min des signed char : -128
max des unsigned char : 255
max des short : 32767
min des short : -32768
max des int : 2147483647
min des int : -2147483648
max des unsigned : 4294967295
```

# Complément à 2

- On voit qu'un type signé sur $n$ octets représente les valeurs entières dans $[-2^{8n-1}; 2^{8n-1} - 1]$.
- Ceci est dû à la représentation en complément à 2.
- Par exemple, pour `signed char`, les valeurs positives sont `00000000`, `00000001`, `00000010`, ... `01111111` (127)
- Et les valeurs strictement négatives sont `11111111` ($-1$), `11111110` ($-2$), ... `10000000` ($-128$)
- On fera dans ce cours l'hypothèse que les entiers signés sont représentés en complément à deux [8].

8. C'est d'ailleurs imposé par le dernier standard C23.

# Types entiers de taille fixe (C99)

- Pour certaines applications, en particulier en réseau, travailler avec des types dont la taille et la « signitude » peuvent varier d'une machine à l'autre est un problème.
- Le standard C99 introduit les types `int8_t`, `uint8_t`, `int16_t`, ..., `uint64_t` dont la taille (c'est le nombre de bits inscrits dans le nom du type) et la signitude (`u` pour `unsigned`, rien pour `signed`) sont fixés.
- Inclure `stdint.h` pour les utiliser.
- Inclure `inttypes.h` pour les macros qui permettent de les afficher/saisir avec `printf`/`scanf`.
- `https://en.wikibooks.org/wiki/C_Programming/inttypes.h`

# Entiers C99 : exemple

```c
#include <stdio.h>
#include <stdint.h> /* types int8_t, etc */
#include <inttypes.h> /* macros pour printf, scanf */

int main(void)
{
    int8_t i = -4;
    printf("taille d'un int8_t : %zu\n", sizeof(int8_t));
    i *= 2;
    printf("i = %"PRIi8"\n", i);
    uint32_t n;
    scanf("%"SCNu32, &n);
    n *= 3;
    printf("n = %"PRIu32"\n", n);
    return 0;
}
```

- Macros `PRI` (pour *print*) ou `SCN` (pour *scan*) suivies de `i` (pour **`int`**, signé) ou `u` (pour **`unsigned`**, non signé) et enfin de la taille en nombre de *bits*.

# Boutisme (*endianness*) : cent-vingt ou vingt et cent ?

- En français, pour compter on utilise principalement la base 10
- et on commence par les plus grandes quantités.
- Par exemple on dit « deux-mille-vingt-six » pour $2 \times 10^3 + 2 \times 10 + 6$.
- Mais on aurait aussi pu dire « six et vingt et deux-mille »

# 2026

- Considérons « deux-mille-vingt-six ».

$$\begin{aligned} 2026_{10} &= 1024 + 512 + 256 + 128 + 64 + 32 + 8 + 2 \\ &= 2^{10} + 2^9 + 2^8 + 2^7 + 2^6 + 2^5 + 2^3 + 2^1 \\ &= 11111101010_2 \\ &= 7ea_{16} \end{aligned}$$

- Par exemple, sur deux octets, on aura `0x07` et `0xea`.
- Ce qui dépend de l'architecture de la machine est l'ordre de ces octets.

# `0xea` et `0x07` ou `0x07` et `0xea` ?

- En 1979, l'architecture Motorola 68000 (un des premiers micro-processeurs grand public) représente l'entier $7ea_{16}$ sur deux octets sous la forme `07ea`, donc commence par les plus grandes quantités.
- Tandis que son concurrent, le micro-processeur Intel 8086 le représente sous la forme `ea07`, donc commence par les plus petites quantités.
- On dit que
  - l'architecture 68000 de Motorola est gros-boutiste (*big-endian*)
  - l'architecture x86 d'Intel est petit-boutiste (*little-endian*).

# Un programme pour afficher les octets de 2026

```c
#include <stdio.h>
#include <inttypes.h>
/* Afficher n octets en hexadécimal à partir de l'adresse start */
void print_bytes(void *start, size_t n);

int main(void)
{
        uint16_t y = 2026;
        uint32_t n = 2026;
        print_bytes(&y, sizeof(y)); printf("\n");
        print_bytes(&n, sizeof(n)); printf("\n");
        return 0;
}

void print_bytes(void *start, size_t n)
{
        uint8_t *byte = start;
        uint8_t *end = byte + n;
        for (byte = start; byte < end; ++byte) {
                printf("%02"PRIx8, *byte);
                if (byte != end - 1)
                        printf(" ");
        }
}
```

# Sortie sur une architecture x86_64

```
$ ./a.out
ea 07
ea 07 00 00
```

# Sortie sur une architecture x86_64

```
$ ./a.out
ea 07
ea 07 00 00
```

La commande `lscpu` permet de connaître, entre autre, le boutisme du processeur.

```
$ lscpu
Architecture :                                    x86_64
  Mode(s) opératoire(s) des processeurs :         32-bit, 64-bit
  Tailles des adresses:                           43 bits physical, 48 bits virtual
  Boutisme :                                      Little Endian
```

# Pourquoi se préoccuper du boutisme ?

- En général, on ne se pose aucune question sur la représentation des entiers en machine et c'est très bien comme ça ! Il vaut mieux travailler au plus haut niveau d'abstraction possible.
- Malheureusement, ces représentations différentes font que deux machines différentes ne peuvent pas, en général, se transmettre des entiers sans précautions préalables.
- Pour cette raison, les protocoles réseau doivent toujours :
  - fixer la taille des entiers échangés ;
  - fixer le boutisme des entiers échangés (en général, sur Internet, gros-boutiste).
- C'est aux programmeurs de faire en sorte que le comportement du programme ne dépende pas de l'architecture de la machine.